Design Patterns

Structural Design Patterns

Saqif Kaisar

Department of CSE, UIU

CSE 3421 – Software Engineering

Structural Patterns – Overview

Purpose

Structural patterns describe how objects and classes can be **combined to form larger, flexible structures**.

There are seven structural patterns:

- **Adapter**
- Bridge
- Composite
- **Decorator**
- Façade
- Flyweight
- Proxy

Focus of This Lecture

Adapter and Decorator patterns.

Adapter Pattern

Main Idea

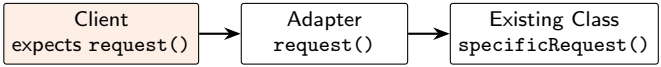
Convert the interface of an existing class into an interface that the client expects.

Incompatible interfaces $\longrightarrow$ **Adapter** $\longrightarrow$ Collaboration

Adapter Pattern – Motivation

The Problem

A useful, pre-existing class cannot be used because its interface does not match the interface required by the client.



Solution

Place an adapter between the client and the existing class. The adapter translates one interface into the other.

Adapter Pattern – Method

The Adapter pattern:

- Helps two incompatible classes or interfaces communicate.
- Assigns interface conversion to a dedicated adapter class.
- Hides incompatibility and translation details from the client.
- Allows useful pre-existing classes to be reused in new code.

Client → Target Interface → Adapter → Adaptee

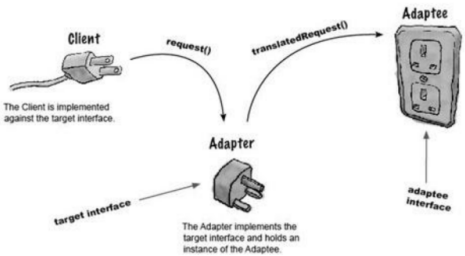
Key Benefit

Reuse an existing implementation without changing either the client or the adapted class.

Adapter Pattern – Plug Example

A traveler has a plug that does not fit the local wall socket.

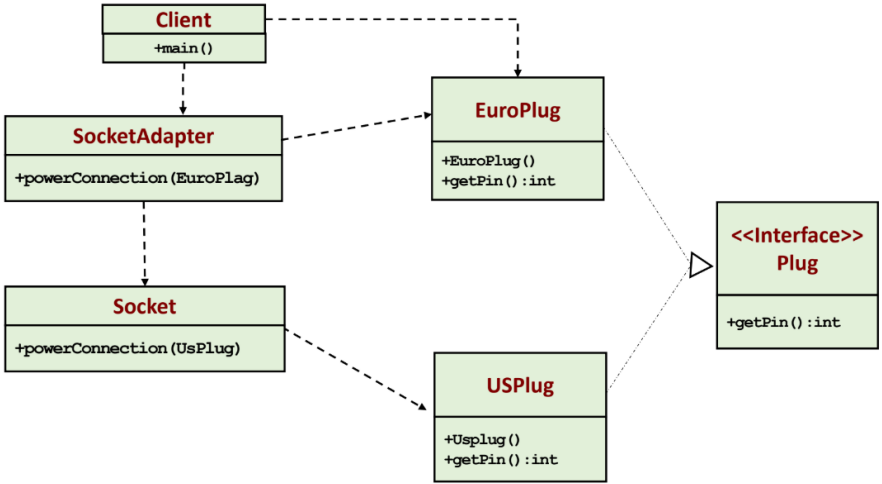
- The plug and socket are both valid.
- Their physical interfaces are incompatible.
- A plug adapter translates between them.



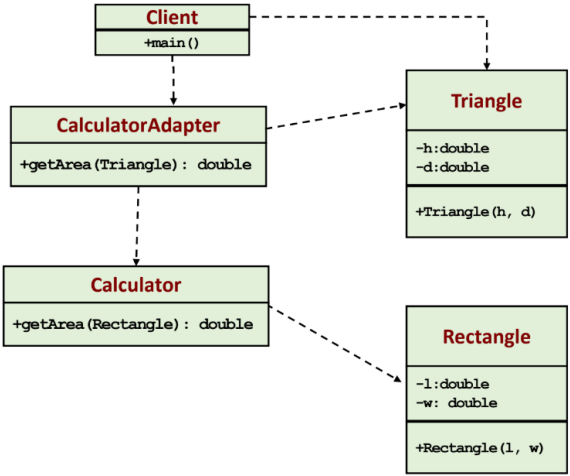
Software Analogy

The adapter preserves the useful device while converting the connection expected by the surrounding system.

Adapter Pattern – Plug UML



Adapter Pattern – Calculator Example



The adapter converts a triangle into dimensions that the existing rectangle-based area calculator can understand.

Adapter Pattern – Practice Problems

Draw UML diagrams for the following scenarios using the Adapter pattern:

1. A video player can play `mp4`, `3gp`, and `vlc` files. The client also wants to play audio files such as `mp3` and `wav` using the same system.

2. A system contains a function that sorts an input array in ascending order. The client wants to use that functionality to find the **second-largest number**.

Hint

Identify the client's expected operation, the reusable existing operation, and the conversion performed by the adapter.

Adapter Pattern – What Problems Does It Solve?

Without an Adapter

- Existing classes have incompatible interfaces.
- Client code must understand conversion details.
- Useful legacy or third-party code may need modification.
- Interface changes create tight coupling.

With an Adapter

- Incompatible components can collaborate.
- Conversion logic stays in one place.
- Existing implementations can be reused unchanged.
- The client depends on the interface it expects.

Adapter solves interface incompatibility.

Decorator Pattern

Main Idea

Add responsibilities to an object dynamically by wrapping it with one or more decorator objects.

Core Object $\rightarrow$ Decorator $\rightarrow$ Enhanced Object

Decorator Pattern – Method

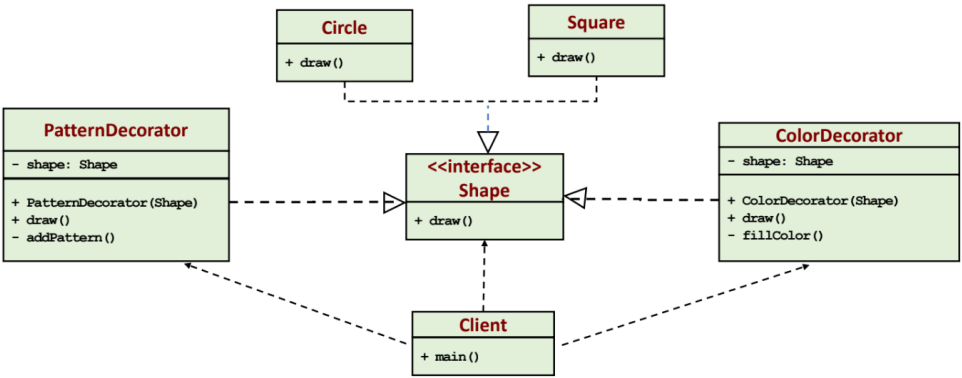
The Decorator pattern:

- Extends existing behavior without modifying the original class.
- Supports the principle **open for extension, closed for modification**.
- Embellishes a core object by recursively wrapping it.
- Works like wrapping a gift, putting it in a box, and wrapping the box.
- Supports a **pay-as-you-go** approach: add only the features needed.

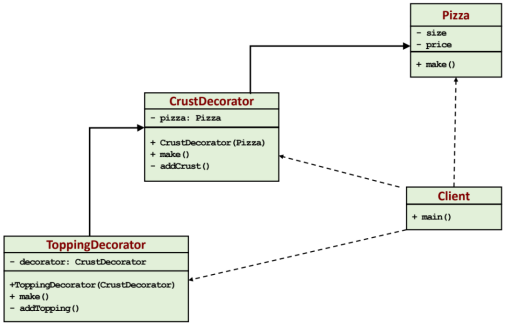
Key Benefit

Responsibilities can be combined at runtime without creating a large hierarchy of subclasses.

Decorator Pattern – Shape UML



Decorator Pattern – Pizza Example



Runtime Composition

Start with a basic pizza, wrap it with a crust decorator, and then wrap the result with a topping decorator.

Decorator Pattern – Practice Problems

Draw UML diagrams for the following scenarios using the Decorator pattern:

1. In presentation software, begin with a basic presentation and add optional features such as annotations, transitions, and animations.

2. In a website builder such as WordPress, begin with a basic template and gradually add fonts, colors, and widgets.

Hint

Model the basic presentation or template as the component, then model each optional feature as a decorator with the same public interface.

Decorator Pattern – What Problems Does It Solve?

Without Decorators

- Every feature combination may require a new subclass.
- The original class must be edited to add behavior.
- Optional features become permanently attached.
- Large inheritance hierarchies become difficult to maintain.

With Decorators

- Behaviors can be added or combined dynamically.
- The original component remains unchanged.
- Clients select only the features they need.
- Small decorators replace many specialized subclasses.

Decorator solves flexible behavior extension.

Structural Patterns – Summary

Pattern	Problem Solved	Core Mechanism
Adapter	Incompatible interfaces prevent reuse	Translate one interface into another
Decorator	An object needs optional, composable behavior	Recursively wrap the object

Key Questions

Adapter: How can these incompatible interfaces work together?

Decorator: How can I add behavior without changing the original class?

Thank You

Questions?